

Lecture - 4 How Java Stores -ve & floating point numbers.

→ float f = 8.125f;

cout(f); → 8.125

System.out.printf("%.20f\n", f); → upto 20 decimal places

↳ 8.12500000000000000000;

but float f = 0.7f;

System.out.printf("%.20f\n", f); → 0.6999999807.....

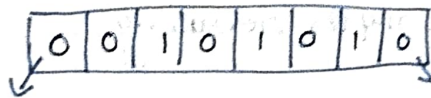
→ but why not exact 0.7 we got.

→ so when we did this cout(0.7) → 0.7 because Java rounded off this value but when we printed this in formatting then we got real value.

* How -ve no. are stored in memory?

byte b = -42; → -128 to 127

First let's see how +42 is stored in memory binary of 42 → 00101010



MSB
(this is always
sign bit)

→ this is how +42 will be stored
LSB but since by default all primitive
types are signed so MSB bit is always
representing sign and rest of bits represents
data.

→ now storing -ve no requires 2's complement

→ Computers only perform binary addition, so negative numbers are stored in a way that subtraction can be done using addition.

Methods to Store -ve Numbers.

- | Method | representation | Problems |
|---------------------|------------------------|---|
| 1) Signed Magnitude | first bit = sign | Two zeros, complex arithmetic |
| 2) One's Complement | Flip all bits | Two zeros, end around carry |
| 3) Two's Complement | Flip all bits
add 1 | No two zeros, no extra hardware, easy arithmetic. |

Why Two's Complement is used

- 1.) Only one zero
- 2.) Subtraction = Addition
- 3.) No special subtraction circuit needed
- 4.) Faster & simpler hardware
- 5.) Handles overflow naturally

* 2's Complement

byte $b = -42$;

→ first ignore sign and convert 42 to binary
⇒ 00101010

→ calculate 1st complement → flip all bits

$$1^{\text{st}} \text{ complement} = 11010101$$

2nd complement = add 1 to 1st complement

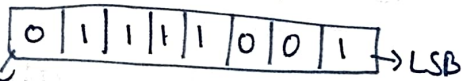
$$\begin{array}{r} 11010101 \\ + 1 \\ \hline 11010110 \end{array}$$

→ this will be stored in memory.

* Now how does it read -ve no

Suppose 11010110 is stored in memory

so it will consider MSB first and since it is 1 so java will treat it as negative no and then will calculate its 2's complement and then convert it to integer that will be 42 and since MSB was 1 so it will become -42.



MSB
(always
sign bit)

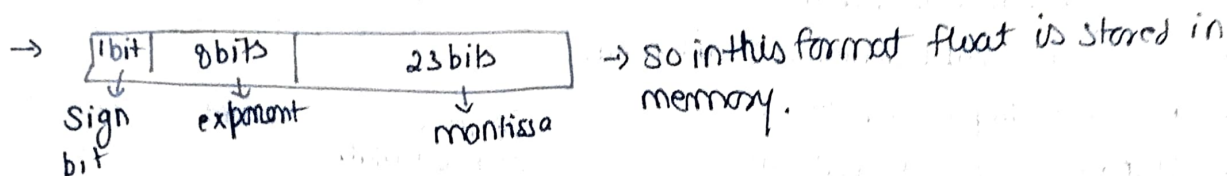
so since byte is 8 bits but 1 bit becomes sign bit so range becomes

→ In case of +ve no MSB will always be 0.

$$[-2^7 \text{ to } 2^7 - 1]$$

* How Floating point numbers are stored as binary?

float $f = 8.125f;$



$f = 8.125f;$

1.) First we convert before & after decimal integers to binary.

$8 \rightarrow 1000$

to calculate after decimal we multiply by 2 till after decimal we get 0

$\rightarrow .125 \times 2 \rightarrow 0.25 \rightarrow 0$
 $\rightarrow .25 \times 2 \rightarrow 0.5 \rightarrow 0$
 $\rightarrow .5 \times 2 \rightarrow 1.0 \rightarrow 1$

binary of $0.125 \rightarrow 001$

$\Rightarrow 1000.001$

2 → Now we will bring it into form $(1.xx) \times 2^{exp}$

$\rightarrow 1.000001 \times 2^3$

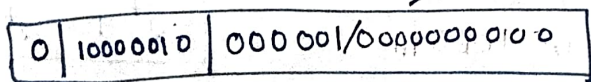
→ if decimal goes left then 2^+
else 2^-

3 → Now we add bias to exponent, for float bias $\Rightarrow 127$

$3 + 127 = 130 \rightarrow$ convert 7 to binary

$130 \rightarrow 1000\ 0010 \rightarrow$ Exponent

4 → Now put value in memory



→ because this is right side part of decimal then we can add many zeroes.

Mantissa is xx part from $1.xx \times 2^{exp}$

* How float binary is read from memory?

Formula $\rightarrow (-1)^{sign} \times (1 + \text{Mantissa}) \times 2^{exp - \text{bias}}$

So while converting binary to decimal

—|—
decimal

So before decimal we will multiply with power of 2 like suppose before decimal it is 1011 $\rightarrow 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$

but after decimal we multiply it by 2^{-1} like if it is 1001

$$\rightarrow 1 \times 2^{-1} + 0 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4}$$

$\rightarrow$ It is simply discrete Maths.

$\rightarrow$ In our case of binary

$$(-1)^0 * (1 + 2^{-6}) * 2^{130-127}$$

$$= \left(1 + \frac{1}{2^6}\right) * 8$$

$$= (1 + 0.015625) * 8$$

$$\Rightarrow 8.125$$

* Case of 0.7f?

$$0.7 \times 2 = 1.4 \rightarrow 1$$

$$0.4 \times 2 = 0.8 \rightarrow 0$$

$$0.8 \times 2 = 1.6 \rightarrow 1$$

$$.6 \times 2 = 1.2 \rightarrow 1$$

$$0.2 \times 2 = 0.4 \rightarrow 0$$

$$0.4 \times 2 = 0.8 \rightarrow 0$$

$$0.8 \times 2 = 1.6 \rightarrow 1$$

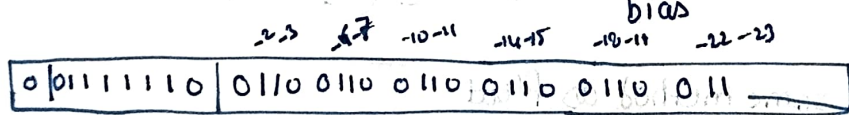
$$\Rightarrow 0.101100110 \dots \dots$$

$$\rightarrow 1.111 \times 2^{exp}$$

$$\Rightarrow (1.01100110 \dots) \times 2^{-1}$$

$$\text{Add} \Rightarrow (1.001100110 \dots) \times 2^{126}$$

See here
pattern is
repeating
infinitely



$\rightarrow$ Sout (f);

$$\rightarrow (-1)^{\text{sign}} \times (1 + \text{Mantissa}) \times 2^{\text{exp} - \text{bias}}$$

$$\Rightarrow (-1)^0 \times (1 + \dots) \times 2^{126-127}$$

$$1 \times \left(1 + \frac{1}{2^1} + \frac{1}{2^2} + \frac{1}{2^3} + \frac{1}{2^4} + \frac{1}{2^5} + \frac{1}{2^6} + \frac{1}{2^7} + \frac{1}{2^8} + \frac{1}{2^9} + \frac{1}{2^{10}} + \frac{1}{2^{11}} + \frac{1}{2^{12}} + \frac{1}{2^{13}} + \frac{1}{2^{14}} + \frac{1}{2^{15}} + \frac{1}{2^{16}} + \frac{1}{2^{17}} + \frac{1}{2^{18}} + \frac{1}{2^{19}} + \frac{1}{2^{20}} + \frac{1}{2^{21}} + \frac{1}{2^{22}} + \frac{1}{2^{23}}\right)$$

$$\Rightarrow \boxed{0.69999998807907104421875} \times 2^{-1}$$

* What is this Bias and why bias = 127 only?

→ Because as per standard of IEEE we don't wanted to store -ve numbers in memory so for that we needed to make exponent +ve. Because storing and processing -ve exponent values will be hardware complex. Now still real exponent can be -ve but we just store it as a +ve value using bias.

- we do this because :-

- No need for sign bit for exponent (stored as unsigned)
- Comparisons become easy (bigger binary = bigger exponent)
- Sorting and arithmetic logic becomes simpler.

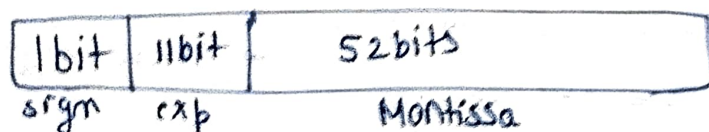
→ why 127 because

exponent has 8 bits → 0 - 255 so we took center point

using IEEE given formula $\Rightarrow (2^{n-1} - 1) \Rightarrow 127$
for simpler calculations and fast calculations

* Expressing double in Memory

double d = 32.1426;



bias becomes

$$\text{bias} = (2^{n-1} - 1) \Rightarrow 1023$$

→ now use some method as float

→ so for both float / double not always it can store exact values. but we can use BigDecimal for exact values.